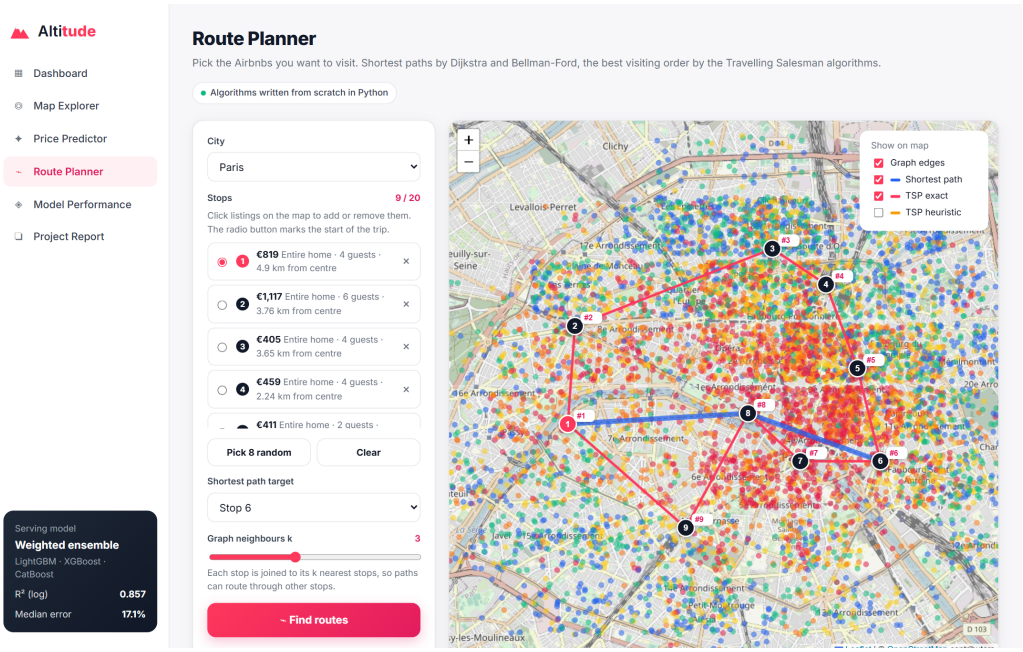
DESIGN AND ANALYSIS OF ALGORITHMS REPORT

Altitude: Route Planning over Airbnb Listings

Shortest paths with Dijkstra and Bellman-Ford, and the Travelling Salesman Problem with Held-Karp dynamic programming and the 2-opt heuristic, implemented from scratch in Python



Subject	Design and Analysis of Algorithms
Algorithms	Dijkstra, Bellman-Ford, Held-Karp, nearest neighbour + 2-opt, DFS, brute force
Verification	11 unit tests against Floyd-Warshall and brute force, all passing
Live application	https://airbnb-price-predictor-ivory.vercel.app
Source code	https://github.com/lovishv1718/airbnb-price-predictor
Date	30 October 2026

Table of Contents

1. Introduction 3

2. Problem Modelling 4

3. Dijkstra's Algorithm 5

4. Bellman-Ford Algorithm 7

5. The Travelling Salesman Problem 8

6. Implementation in the Application 11

7. Testing and Verification 13

8. Empirical Analysis 14

9. Comparison and Conclusion 15

1. Introduction

Altitude is a web application about Airbnb listings in ten European cities. Its Route Planner answers two classic algorithmic questions for a traveller who wants to visit several listings, for example to view apartments before booking:

- **Shortest path:** what is the shortest way from my starting listing to each other listing, moving along a network that connects nearby listings?
- **Travelling Salesman Problem (TSP):** in which order should I visit all the chosen listings, and return to the start, to travel the least distance?

The first is solved in polynomial time by two different paradigms, greedy (Dijkstra) and dynamic programming (Bellman-Ford). The second is NP-hard. It is solved exactly by exponential-time dynamic programming (Held-Karp) and approximately by a fast greedy-plus-local-search heuristic (nearest neighbour + 2-opt). Every algorithm is implemented from scratch in `src/algorithms.py`, using only the standard library's `heapq` for the priority queue, and served to the browser through POST `/api/route`.

1.1 Objectives

- Model the trip-planning problem as a weighted graph.
- Implement Dijkstra, Bellman-Ford, Held-Karp and 2-opt, and state and justify their complexity.
- Prove or argue correctness, and verify it with tests against independent reference algorithms.
- Measure run times empirically and compare them with the theoretical growth rates.
- Make the algorithms interactive in a web interface on real listing locations.

1.2 Algorithm design paradigms covered

Paradigm	Algorithm in this project	Problem
Greedy	Dijkstra; nearest-neighbour tour	Shortest path; TSP construction
Dynamic programming	Bellman-Ford; Held-Karp	Shortest path; exact TSP
Graph traversal	Depth-first search	Connected components
Local search	2-opt	TSP improvement
Brute force	All permutations (used in tests)	Exact TSP reference
Priority queue	Binary min-heap (heapq)	Dijkstra's frontier

2. Problem Modelling

2.1 Graph definition

Let the n listings selected by the user be the vertex set V. The weight of an edge (u, v) is the great-circle distance between the two listings, from the haversine formula:

$$d(u, v) = 2R \cdot \operatorname{asin}\left(\sqrt{ \sin^2(\Delta\phi/2) + \cos\phi_u \cdot \cos\phi_v \cdot \sin^2(\Delta\lambda/2) } \right), \quad R = 6371 \text{ km}$$

A complete graph would make every shortest path a single direct edge, which is uninteresting. Instead each vertex is joined to its **k nearest neighbours** (default k = 3), which imitates a sparse street network. The graph is undirected and $|E| \leq k \cdot n$. Because a k-nearest-neighbour graph can be disconnected, **depth-first search** finds the connected components, and the closest pair of vertices between the first component and the rest is joined until only one component remains.

```
def build_graph(points, k=3):
    """Undirected weighted graph {node: {neighbour: km}} linking every point to its
    k nearest neighbours. Disconnected components are then joined by their
    closest pair of points, so every listing is reachable."""
    n = len(points)
    graph = {i: {} for i in range(n)}

    def connect(i, j):
        w = haversine_km(points[i], points[j])
        graph[i][j] = graph[j][i] = w

    for i in range(n):
        nearest = sorted((haversine_km(points[i], points[j]), j) for j in range(n) if j != i)
        for _, j in nearest[:k]:
            connect(i, j)

    while True:
        comps = components(graph)
        if len(comps) <= 1:
            break
        a = comps[0]
        rest = [v for c in comps[1:] for v in c]
        _, i, j = min((haversine_km(points[i], points[j]), i, j) for i in a for j in rest)
        connect(i, j)
    ...
```

Listing 2.1 - Building the k-nearest-neighbour graph (adjacency dictionary {u: {v: km}}).

Complexity. For each of the n vertices the other n − 1 distances are sorted: $O(n^2 \log n)$. Each DFS is O(V + E), and joining components takes at most $O(n^2)$ per join. The adjacency dictionary uses O(V + E) space. With $n \leq 20$ in the app, construction is negligible.

2.2 Representation

The graph is a dictionary of dictionaries: `graph[u][v] = weight`. This gives O(1) average edge lookup, iterates neighbours in O(deg u), and stores each undirected edge in both directions. For the TSP, an n × n matrix of shortest-path distances is computed by running Dijkstra from every vertex. This is the *metric closure*, so every leg of a tour follows a real path in the graph.

3. Dijkstra's Algorithm

3.1 Idea and pseudocode

Dijkstra's algorithm grows a set of *settled* vertices whose shortest distance from the source s is final. It repeatedly settles the unsettled vertex with the smallest tentative distance, then *relaxes* its outgoing edges: if going through u gives v a shorter distance, the distance is updated. A binary min-heap holds the frontier. Instead of a decrease-key operation, a new entry is pushed and stale entries are skipped when popped (lazy deletion).

```
Dijkstra(G, s):
    dist[v] = INF for all v;  dist[s] = 0;  heap = [(0, s)]
    while heap not empty:
        (d, u) = pop_min(heap)
        if u is settled: continue                # stale entry
        settle u
        for each edge (u, v, w):
            if d + w < dist[v]:
                dist[v] = d + w;  prev[v] = u;  push(heap, (dist[v], v))
```

3.2 Correctness

Claim: when a vertex u is settled, dist[u] equals the true shortest distance δ(s, u). **Argument (exchange):** suppose u is the first vertex settled with dist[u] > δ(s, u). Take a shortest path from s to u, and let y be its first vertex that is not yet settled, with x settled just before it. Since x was settled correctly and the edge (x, y) was relaxed, dist[y] ≤ δ(s, x) + w(x, y) = δ(s, y) ≤ δ(s, u) < dist[u]. With non-negative weights, the heap would therefore pop y before u, a contradiction. The argument needs non-negative weights, which holds for distances.

3.3 Complexity

Each vertex is settled once. Each edge is relaxed at most once per direction and pushes at most one heap entry, so the heap holds O(E) entries and every push and pop costs O(log E) = O(log V). The total is **O((V + E) log V)** time and O(V + E) space. With a Fibonacci heap this would improve to O(E + V log V), but the binary heap is simpler and faster in practice for small graphs.

3.4 Trace on a 4-vertex graph

Edges: A-B 4, A-C 1, B-C 2, B-D 1, C-D 5; source A. The direct edge A-B (4) is beaten by A-C-B (3), and A-D comes out as A-C-B-D (4).

Step	Action	A	B	C	D	Heap after step
1	settle A (d = 0)	0	4	1	∞	(1,C), (4,B)
2	settle C (d = 1)	0	3	1	6	(3,B), (4,B), (6,D)
3	settle B (d = 3)	0	3	1	4	(4,B), (4,D), (6,D)
-	(4, B) stale, skipped					
4	settle D (d = 4)	0	3	1	4	(6,D)
-	(6, D) stale, skipped					

Final distances: A = 0, B = 3, C = 1, D = 4. Shortest path to D: A → C → B → D. The entries (4, B) and (6, D) went stale when B and D found shorter distances; they are skipped when popped, which is lazy deletion at work.

3.5 Implementation

```
def dijkstra(graph, source):
    """Dijkstra's algorithm with a binary min-heap and lazy deletion."""
    t = time.perf_counter()
    dist = {v: INF for v in graph}
    prev = {v: None for v in graph}
    dist[source] = 0.0
    heap = [(0.0, source)]
    done = set()
    order, relaxations, pushes = [], 0, 1
    while heap:
        d, u = heapq.heappop(heap)
        if u in done:  # stale heap entry
            continue
        done.add(u)
        order.append(u)
        for v, w in graph[u].items():
            relaxations += 1
            if d + w < dist[v]:
                dist[v] = d + w
                prev[v] = u
                heapq.heappush(heap, (dist[v], v))
                pushes += 1
    return dist, prev, {"visit_order": order, "relaxations": relaxations, "heap_pushes": pushes,
                        "ms": (time.perf_counter() - t) * 1000}
```

Listing 3.1 - Dijkstra with heapq and lazy deletion; it also counts relaxations and heap pushes for the analysis.

4. Bellman-Ford Algorithm

4.1 Idea and pseudocode

Bellman-Ford relaxes *every* edge, in a fixed order, in repeated passes. After pass i, every shortest path that uses at most i edges has been found. A shortest path in a graph without negative cycles has at most V − 1 edges, so V − 1 passes suffice. One extra pass detects a negative cycle: if any edge can still be relaxed, distances can decrease forever. This implementation also stops early when a pass changes nothing.

```
Bellman-Ford(G, s):
    dist[v] = INF for all v;  dist[s] = 0
    repeat V - 1 times:
        changed = false
        for each directed edge (u, v, w):
            if dist[u] + w < dist[v]:  dist[v] = dist[u] + w;  prev[v] = u;  changed = true
        if not changed: break                          # early exit
    negative_cycle = exists (u, v, w) with dist[u] + w < dist[v]
```

4.2 Correctness and complexity

Path-relaxation property: if v0 = s, v1, ..., vk is a shortest path and its edges are relaxed in that order (with any other relaxations in between), then dist[vk] = δ(s, vk). Pass i relaxes all edges, so it relaxes the i-th edge of every shortest path, and by induction all shortest paths of at most i edges are correct after pass i. Each pass costs O(E), and there are at most V − 1 passes plus one check, so the total is **O(V · E)** time and O(V) extra space. Unlike Dijkstra, the algorithm is correct with negative edge weights, which is why it is the basis of distance-vector routing.

4.3 Trace on the same graph

Edge order per pass: A→B, A→C, B→A, B→C, B→D, C→A, C→B, C→D, D→B, D→C.

Pass	A	B	C	D	Edges that improved a distance
0 (init)	0	∞	∞	∞	-
1	0	3	1	5	A→B, A→C, B→D, C→B
2	0	3	1	4	B→D
3	0	3	1	4	no change: stop

In pass 1, B→D sets D to 5 using B's old distance of 4; C→B then lowers B to 3, but only after B→D has already been relaxed. D therefore reaches its true distance of 4 (route A-C-B-D, three edges) in pass 2. Pass 3 changes nothing, so the loop stops after 3 of 3 allowed passes, with the same distances as Dijkstra. The unit tests also check a graph with a negative edge (Bellman-Ford returns the correct distance) and a negative cycle (detected).

```
def bellman_ford(graph, source):
    """Bellman-Ford: relax every directed edge up to V-1 times, stopping early once a
    full pass changes nothing, then run one more pass to detect negative cycles."""
    t = time.perf_counter()
    edges = [(u, v, w) for u in graph for v, w in graph[u].items()]  # both directions
    dist = {v: INF for v in graph}
    prev = {v: None for v in graph}
    dist[source] = 0.0
    passes, relaxations = 0, 0
    for _ in range(len(graph) - 1):
        passes += 1
        changed = False
        for u, v, w in edges:
            relaxations += 1
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                changed = True
        if not changed:
            break
    negative_cycle = any(dist[u] + w < dist[v] for u, v, w in edges)
    return dist, prev, {"passes": passes, "max_passes": len(graph) - 1, "relaxations": relaxations,
                        "directed_edges": len(edges), "negative_cycle": negative_cycle,
                        "ms": (time.perf_counter() - t) * 1000}
```

Listing 4.1 - Bellman-Ford with early exit and negative-cycle detection.

5. The Travelling Salesman Problem

Given n locations and the distance between every pair, the TSP asks for the shortest closed tour that visits each location exactly once. Its decision version is NP-complete, so no polynomial-time algorithm is known. The number of distinct tours of n locations is (n − 1)!/2, which makes brute force hopeless beyond about 12 locations:

n	Tours (n−1)!/2	Brute force ~n!	Held-Karp ~$n^2 \cdot 2^n$
5	12	1.20e+02	800
8	2,520	4.03e+04	16,384
10	181,440	3.63e+06	102,400
12	19,958,400	4.79e+08	589,824
15	43,589,145,600	1.31e+12	7,372,800
20	60,822,550,204,416,000	2.43e+18	419,430,400

5.1 Held-Karp dynamic programming

Held-Karp avoids enumerating every order by noticing that the best way to finish a tour depends only on *which* locations have been visited and *where* you are now, not on the order so far. Let C(S, j) be the length of the shortest path that starts at the start vertex, visits exactly the set S, and ends at j S. Then:

```
C({j}, j)    = D[start][j]
C(S + k, k)  = min over j in S of  C(S, j) + D[j][k]          for k not in S
optimum      = min over j of  C(all, j) + D[j][start]
```

Optimal substructure: the last edge of an optimal path ending at k comes from some j, and the part before it must itself be optimal for (S, j); otherwise it could be replaced by a shorter one. **Complexity:**

there are 2^{n-1} subsets and n end vertices, so $O(n \cdot 2^n)$ states, and each state takes a minimum over n predecessors: **$O(n^2 \cdot 2^n)$ time and $O(n \cdot 2^n)$ space**. The subsets are stored as integer bitmasks, so set membership and union are single bit operations.

Trace on four cities 0-3 with distances 0-1 = 2, 0-2 = 9, 0-3 = 10, 1-2 = 6, 1-3 = 4, 2-3 = 8 (start 0). Each row is one DP state, with the predecessor that achieved it:

Subset S	End j	C(S, j)	Came from
{1}	1	2	0
{2}	2	9	0
{3}	3	10	0
{1,2}	1	15	2
{1,2}	2	8	1
{1,3}	1	14	3
{1,3}	3	6	1
{2,3}	2	18	3
{2,3}	3	17	2
{1,2,3}	1	21	3
{1,2,3}	2	14	3
{1,2,3}	3	16	2

Closing the tour: min over j of C({1,2,3}, j) + D[j][0] = 23. The optimal tour is 0 → 2 → 3 → 1 → 0 with length 23, the same as brute force.

```
def tsp_held_karp(D, start=0):
    """Exact TSP by Held-Karp dynamic programming over subsets (bitmasks).
    C[mask][j] = shortest path that starts at `start`, visits exactly the nodes in
    mask and ends at j. Returns a closed tour start -> ... -> start."""
    t = time.perf_counter()
    n = len(D)
    if n == 1:
        return [start, start], 0.0, {"states": 1, "ms": 0.0}
    others = [v for v in range(n) if v != start]
    m = len(others)
    full = (1 << m) - 1
    C = [[INF] * m for _ in range(1 << m)]
    parent = [[-1] * m for _ in range(1 << m)]
    for j in range(m):
        C[1 << j][j] = D[start][others[j]]
    states = 0
    for mask in range(1, full + 1):
        for j in range(m):
            if not mask & (1 << j) or C[mask][j] == INF:
                continue
            for k in range(m):
                if mask & (1 << k):
                    continue
                states += 1
                nm = mask | (1 << k)
                cand = C[mask][j] + D[others[j]][others[k]]
                if cand < C[nm][k]:
                    C[nm][k] = cand
                    parent[nm][k] = j
    best, last = min((C[full][j] + D[others[j]][start], j) for j in range(m))
    order, mask = [], full
    while last != -1:
        order.append(others[last])
        last, mask = parent[mask][last], mask & ~(1 << last)
    tour = [start] + order[::-1] + [start]
    return tour, best, {"states": states, "subsets": 1 << m, "ms": (time.perf_counter() - t) * 1000}
```

Listing 5.1 - Held-Karp over bitmask subsets with parent pointers to rebuild the tour.

5.2 Nearest neighbour + 2-opt heuristic

Nearest neighbour (greedy) always moves to the closest unvisited location: O(n²), but it often leaves long edges and crossings near the end. **2-opt** (local search) then looks at every pair of edges (a, b) and (c, d). If D[a][c] + D[b][d] < D[a][b] + D[c][d], it reverses the segment between them, which removes a crossing. It repeats until no improving move exists. Each pass checks O(n²) pairs. The result is a *local* optimum: no single 2-opt move helps, but the tour is not guaranteed to be globally optimal.

```
def two_opt(tour, D):
    """2-opt local search: reverse any segment that shortens the tour, until no
    improving move is left."""
    tour = tour[:]
    improved, passes, swaps = True, 0, 0
    while improved:
        improved = False
        passes += 1
        for i in range(1, len(tour) - 2):
            for j in range(i + 1, len(tour) - 1):
                a, b, c, d = tour[i - 1], tour[i], tour[j], tour[j + 1]
                if D[a][c] + D[b][d] < D[a][b] + D[c][d] - 1e-12:
                    tour[i:j + 1] = reversed(tour[i:j + 1])
                    improved = True
                    swaps += 1
    return tour, {"passes": passes, "swaps": swaps}
```

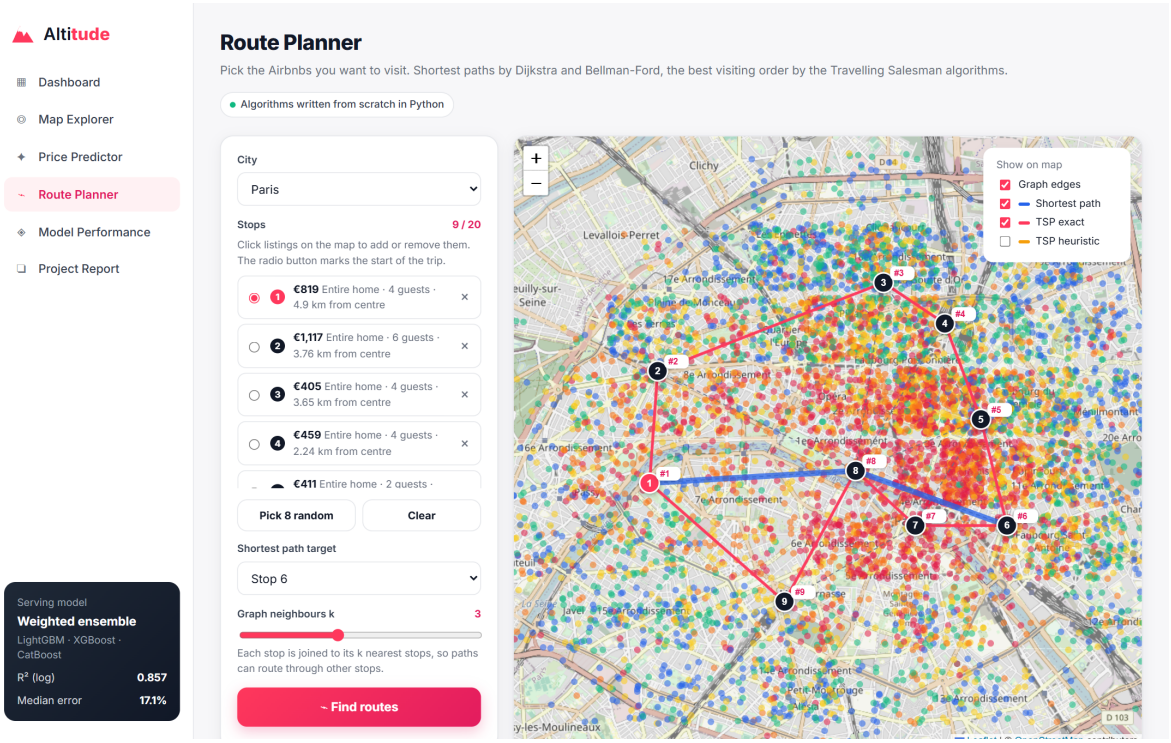
Listing 5.2 - 2-opt local search.

6. Implementation in the Application

In the Route Planner page the user picks a city and clicks up to 20 listings on the map. The browser sends their coordinates, the start, the target and k to POST `/api/route`. `plan_routes()` builds the graph and runs Dijkstra and Bellman-Ford from the start, Dijkstra from every vertex for the distance matrix, the heuristic, and Held-Karp when n ≤ 12. It returns paths, tours, distances, operation counts and timings as JSON.

6.1 Worked example: nine Paris landmarks

The example uses the Paris listing closest to each of nine landmarks, starting at the Eiffel Tower, with Bastille as the shortest-path target (k = 3: 9 vertices, 18 edges).



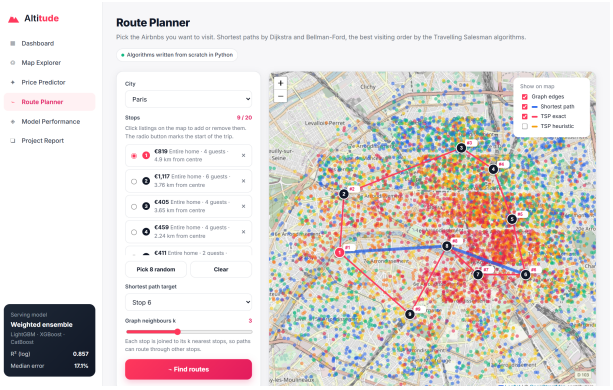
Screenshot 6.1 - Exact TSP tour (red), Dijkstra path to Bastille (blue), graph edges (dashed) and numbered stops.

Stop	Near	Dijkstra km	Bellman-Ford km	Shortest path
1	Eiffel Tower	0.000	0.000	1
2	Arc de Triomphe	1.725	1.725	1 → 2
3	Sacré-Cœur	5.446	5.446	1 → 2 → 3
4	Gare du Nord	5.827	5.827	1 → 8 → 4
5	République	5.251	5.251	1 → 8 → 5
6	Bastille	5.649	5.649	1 → 8 → 6
7	Notre-Dame	4.417	4.417	1 → 8 → 7
8	Louvre	3.180	3.180	1 → 8

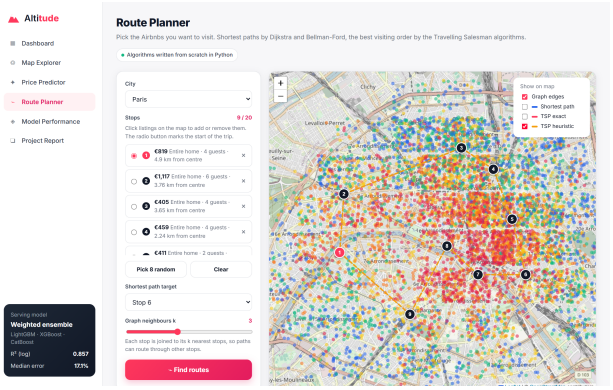
Stop	Near	Dijkstra km	Bellman-Ford km	Shortest path
9	Montparnasse	2.765	2.765	1 → 9

Algorithm	Result	Route	Work done
Dijkstra	5.65 km	1 → 8 → 6	36 relaxations, 10 heap pushes
Bellman-Ford	5.65 km	1 → 8 → 6	2 of 8 passes, 72 relaxations
Held-Karp	17.52 km	1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 1	256 subsets, 3,584 transitions
NN + 2-opt	17.52 km	1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 1	greedy 20.14 km, 1 swap(s)

Both shortest-path algorithms agree on every stop. The route to Bastille goes through the Louvre listing (5.65 km) because the two have no direct edge. The greedy tour 1 → 2 → 8 → 7 → 6 → 5 → 4 → 3 → 9 → 1 was 20.14 km, 15% longer than optimal; one 2-opt swap removed its crossing and reached the 17.52 km optimum found by Held-Karp.

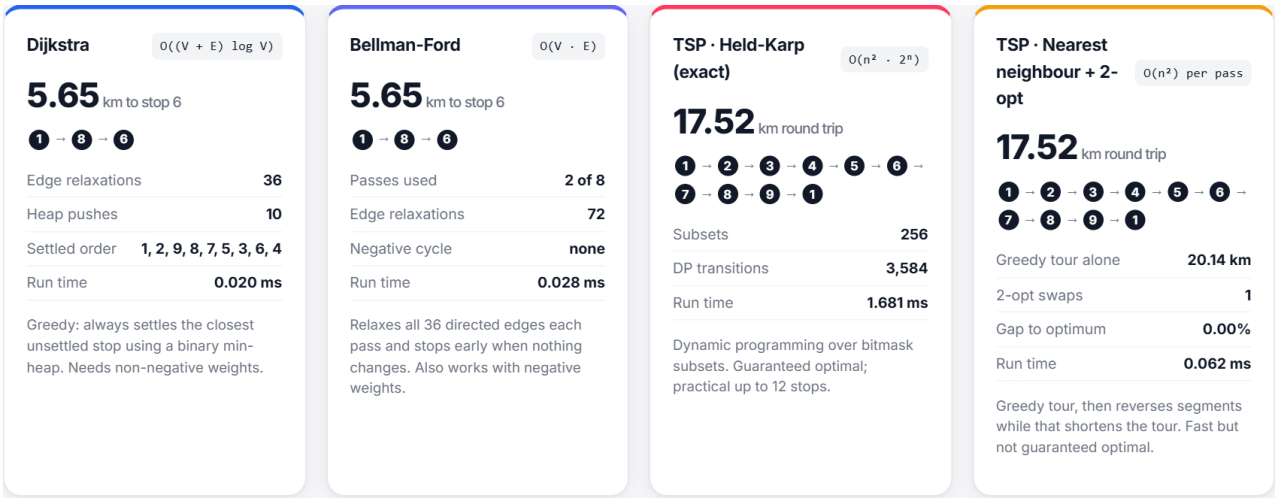


Held-Karp exact tour



Nearest neighbour + 2-opt tour

Screenshot 6.2 - The two TSP tours drawn on the map.



Screenshot 6.3 - Result cards: complexity, route, operation counts and run time for each algorithm.



Shortest distances from stop 1

Dijkstra and Bellman-Ford must agree on every stop: **they do ✓**

STOP	DIJKSTRA KM	BELLMAN-FORD KM	PATH
1	0.000	0.000	1
2	1.725	1.725	1 → 2
3	5.446	5.446	1 → 2 → 3
4	5.827	5.827	1 → 8 → 4
5	5.251	5.251	1 → 8 → 5
6	5.649	5.649	1 → 8 → 6
7	4.417	4.417	1 → 8 → 7
8	3.180	3.180	1 → 8
9	2.765	2.765	1 → 9

All-pairs shortest-path matrix (km)

Dijkstra run from every stop; this is the distance matrix the TSP solvers use

	1	2	3	4	5	6	7	8	9
1	–	1.73	5.45	5.83	5.25	5.65	4.42	3.18	2.77
2	1.73	–	3.72	4.85	5.48	5.88	4.64	3.41	4.49
3	5.45	3.72	–	1.13	2.58	4.26	4.17	2.93	5.21
4	5.83	4.85	1.13	–	1.57	3.25	3.48	2.65	4.93
5	5.25	5.48	2.58	1.57	–	1.68	1.91	2.07	4.24
6	5.65	5.88	4.26	3.25	1.68	–	1.41	2.47	3.74
7	4.42	4.64	4.17	3.48	1.91	1.41	–	1.24	2.33
8	3.18	3.41	2.93	2.65	2.07	2.47	1.24	–	2.28
9	2.77	4.49	5.21	4.93	4.24	3.74	2.33	2.28	–

Screenshot 6.4 - Cross-check of the two shortest-path algorithms, and the all-pairs distance matrix used by the TSP.

7. Testing and Verification

`tests/test_algorithms.py` has 11 `unittest` tests, all passing. The key idea is to compare each algorithm with an independent reference that is simple enough to be obviously correct:

Test	Reference	Inputs
Dijkstra and Bellman-Ford distances	Floyd-Warshall $O(V^3)$	30 random graphs, every source
Paths are valid and sum to the distance	Edge-by-edge check	random graph
Hand-worked example (Chapter 3)	Known answer A-C-B-D = 4	4 vertices
Negative edge, negative cycle	Known answers	small directed graphs
Held-Karp is optimal	Brute force O(n!)	15 instances, 3-8 stops
2-opt never beats the optimum	Held-Karp	15 instances, 9 stops
Tours visit every stop once	Sorted-set check	both TSP solvers
Walks use real edges; graph connected and symmetric	DFS, adjacency check	random graphs
Planner summary; exact TSP skipped above the limit	plan_routes()	8 and 15 stops

```
def test_held_karp_matches_brute_force(self):
    for seed in range(15):
        n = 3 + seed % 6  # 3..8 nodes
        D, _ = all_pairs(build_graph(random_points(n, seed), k=2))
        _, hk = tsp_held_karp(D, 0)[:2]
        _, bf = tsp_brute_force(D, 0)
        self.assertAlmostEqual(hk, bf, places=9)
```

Listing 7.1 - Exact TSP checked against brute force.

8. Empirical Analysis

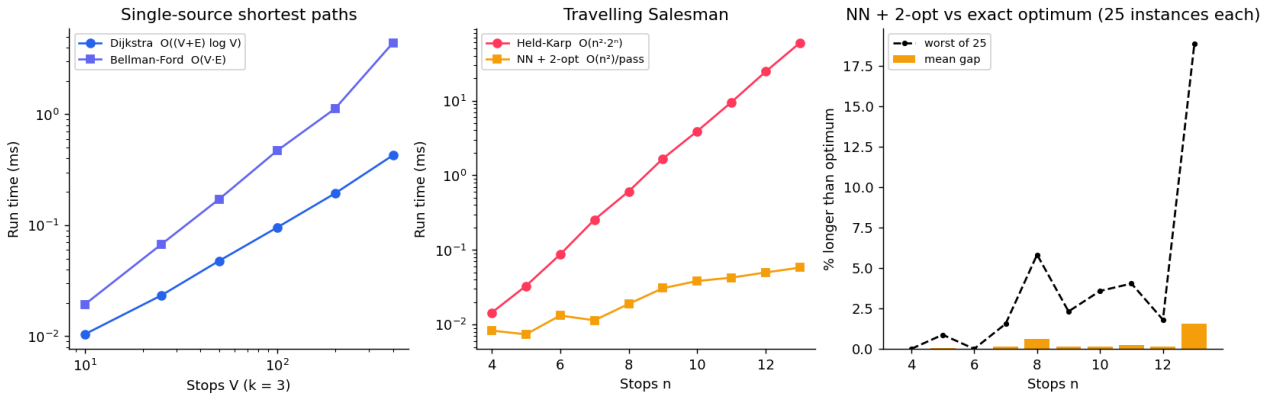


Figure 8.1 - Measured run times (log scale) and the 2-opt optimality gap on random Paris-sized instances.

Stops V	Edges E	Dijkstra ms	Bellman-Ford ms	Ratio
10	22	0.010	0.019	1.9×
25	47	0.023	0.067	2.9×
50	92	0.048	0.172	3.6×
100	185	0.095	0.470	4.9×
200	378	0.194	1.124	5.8×
400	751	0.427	4.409	10.3×

Shortest paths. With k = 3, E ≈ 2V, so Dijkstra's O((V + E) log V) is close to linear, and the measurements grow almost in proportion to V. Bellman-Ford is 10× slower at 400 stops and the gap widens as V grows. Thanks to the early exit it needs far fewer than V − 1 passes on these geometric graphs, so it stays well below its O(V · E) worst case, but every pass still scans every edge.

Stops n	4	5	6	7	8	9	10	11	12	13
Held-Karp ms	0.01	0.03	0.09	0.25	0.61	1.67	3.90	9.55	24.51	59.55
NN + 2-opt ms	0.01	0.01	0.01	0.01	0.02	0.03	0.04	0.04	0.05	0.06
Mean gap %	0.00	0.03	0.00	0.15	0.61	0.12	0.15	0.26	0.12	1.57
Worst gap %	0.0	0.9	0.0	1.5	5.8	2.3	3.6	4.0	1.8	18.9
Optimal in %	100	96	100	88	68	92	88	88	88	80

TSP. Theory predicts that Held-Karp's time grows by $2\cdot((n+1)/n)^2$ per extra stop, about 2.3-2.5 for these sizes. The measured factor is 2.5×, reaching 60 ms at 13 stops. Extrapolating, 20 stops would take about 37 seconds in pure Python and need 10 million DP entries in memory, and every extra stop doubles that again. This is why the app switches to the heuristic above 12 stops, so every request stays instant.

Nearest neighbour + 2-opt stays below 0.1 ms. Across 25 random instances per size it found the exact optimum in 68-100% of cases, with a mean gap below 2%. Its worst case, 19% longer than optimal, shows the price of giving up the optimality guarantee.

9. Comparison and Conclusion

Algorithm	Paradigm	Time	Space	Optimal?	Negative weights
Dijkstra	Greedy	O((V+E) log V)	O(V+E)	Yes	No
Bellman-Ford	Dynamic programming	O(V·E)	O(V)	Yes	Yes, detects cycles
Held-Karp	Dynamic programming	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$	Yes	n/a
NN + 2-opt	Greedy + local search	$O(n^2)$ per pass	O(n)	No (local optimum)	n/a
Brute force	Exhaustive	O(n!)	O(n)	Yes	n/a

The project shows the central trade-offs of algorithm design on a real problem. For shortest paths, the greedy algorithm wins whenever its precondition, non-negative weights, holds. The more general dynamic-programming algorithm pays for its generality with an extra factor of roughly V. For the TSP, exact dynamic programming turns an O(n!) search into $O(n^2 \cdot 2^n)$. That is a huge saving, yet still exponential, so a polynomial heuristic is essential beyond a dozen stops. In practice the heuristic was optimal most of the time, but not always. Each claim was checked twice: the unit tests confirm correctness against independent references, and the benchmark confirms the predicted growth rates.

Limitations and future work

- Edge weights are straight-line distances; a real street graph (e.g. OpenStreetMap) would give walking or driving distances.
- Further algorithms that fit the same application: Prim/Kruskal minimum spanning trees with union-find, 0/1 knapsack for choosing listings within a budget, branch and bound for the TSP, and A* search for point-to-point paths.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2022). *Introduction to Algorithms*, 4th ed. MIT Press.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269-271.
- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87-90.
- Held, M. and Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the SIAM*, 10(1), 196-210.
- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791-812.
- Kleinberg, J. and Tardos, É. (2005). *Algorithm Design*. Pearson.